ALICIA VÁZQUEZ

Sincronía y API Rest

Alicia Vázquez
@aliciaFPInf

CC BY NC SA

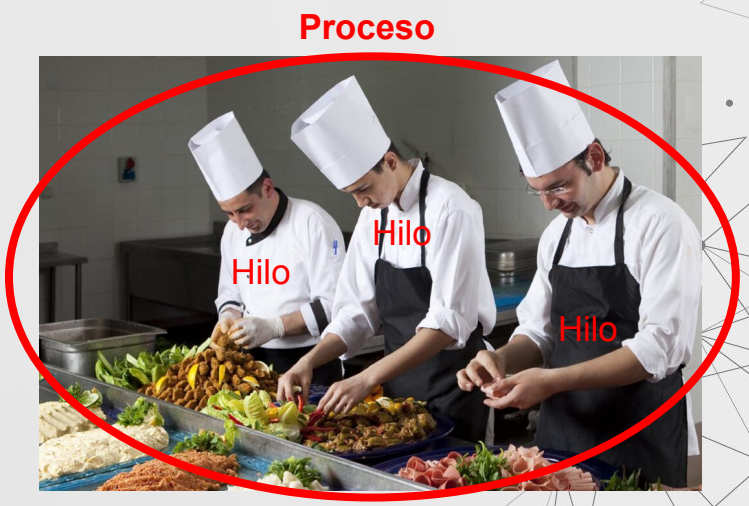
01

Hilos y proceso

¿Qué es un Proceso y un Hilo de Ejecución?

Entender cómo funcionan los procesos e hilos en un programa y su relación con la sincronía y asincronía.

- Un **proceso** es un programa en ejecución con **su propia memoria**.
- Un **hilo** es una unidad de ejecución dentro de un proceso y c**omparte memoria** con otros hilos.
- Los procesos son más pesados que los hilos, pero los hilos requieren sincronización para evitar errores.



Proceso

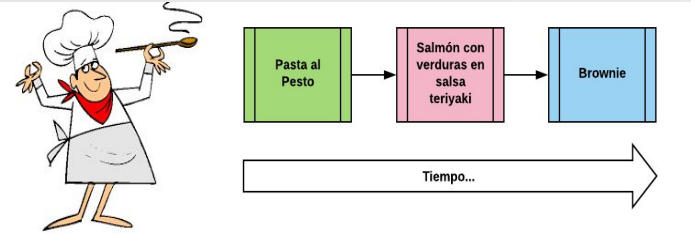
Se define como un programa en ejecución con su propio **espacio de memoria y recursos**.

Imagínalo como una mesa de un restaurante con su propia orden y clientes.

Hilo de Ejecución

Es la secuencia de instrucciones que sigue el proceso.

Si hay un solo cocinero para preparar todos los platos uno tras otro, hablamos de un hilo único.



Imagina un cocinero en la cocina de un restaurante. Si hay solo un cocinero, debe preparar cada plato en orden, uno tras otro. Esto representa un hilo único.

El modelo de JavaScript:

JS utiliza un **único hilo de ejecución**, pero gestiona tareas asíncronas mediante el **Event Loop**.

02

Sincróno o asincróno

Para saber más: Lemon Code

Sincronía (el bloqueo)

Un **proceso síncrono** ejecuta tareas una tras otra, sin avanzar hasta que la tarea actual se complete.

JavaScript ejecuta las instrucciones en orden, una tras otra, de forma **síncrona** por defecto.

```
console.log("Primer mensaje");
console.log("Segundo mensaje");
console.log("Tercer mensaje");
```

Un cajero en un supermercado con una sola fila. Cada cliente es atendido uno por uno, y el siguiente solo puede pasar cuando el anterior ha terminado.

Asincronía (fluidez)

Un **proceso asíncrono** permite que el programa continúe con otras tareas mientras espera que una operación termine.

```
console.log("Inicio");

setTimeout(() => {
   console.log("Esto se ejecuta después de 2
segundos");
}, 2000);

console.log("Fin");
```

Se imprime "Inicio", luego "Fin", y después de 2 segundos, "Esto se ejecuta después de 2 segundos". JavaScript sigue ejecutando código mientras espera el **setTimeout**.

Un restaurante con varios camareros. Si un cliente pide un plato que tarda mucho, el camarero puede seguir atendiendo otras mesas mientras la comida se prepara.

Diferencia entre Asincronía y Sincronía

Característica	Sincrónico	Asincrónico
Ejemplo real	Un cajero con una sola fila	Un restaurante con varios camareros
Ejecución	En orden, bloquea el siguiente paso hasta terminar	Puede seguir ejecutando tareas mientras espera
Ejemplo código	`console.log("Hola");` `console.log("Mundo");`	`setTimeout(() =>` `console.log("Mundo"), 2000);`

Event Loop de JS

JavaScript usa un único hilo de ejecución, pero puede manejar tareas **asíncronas** con el **Event Loop**.

El **Event Loop** es un mecanismo que permite la ejecución de código **asíncrono** en un lenguaje que es **monohilo** (single-threaded). Su función principal es gestionar la ejecución de tareas en la pila de ejecución (**Call Stack**) y en la cola de tareas (**Task Queue** o **Callback Queue**).

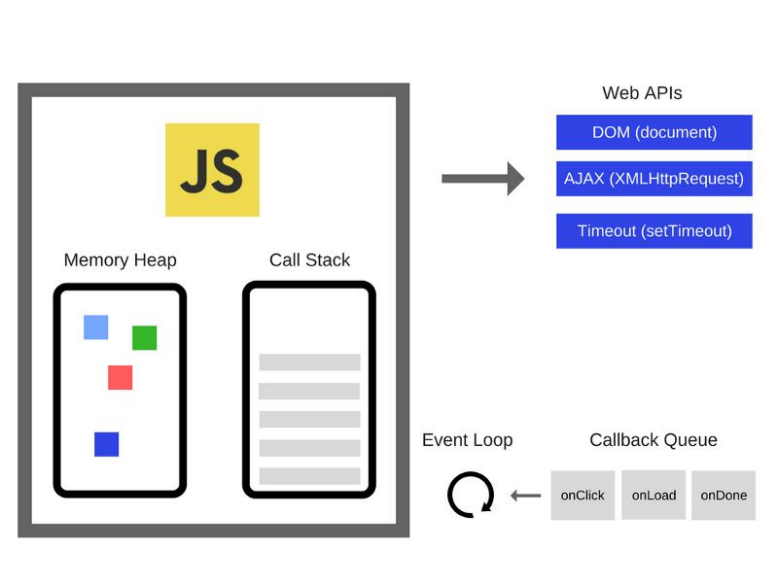
Beneficios del **Event Loop**

1. Permite que JavaScript maneje **múltiples tareas** sin bloquear la ejecución.
2. Hace posible la programación **asíncrona** con setTimeout, *fetch, Promesas, async/await*, etc.
3. Evita que la UI del navegador se congele cuando JavaScript ejecuta tareas largas.

Un ejemplo real de esto es el Scroll Infinito.

Funcionamiento del Event Loop

1. JavaScript ejecuta código de forma síncrona en el Call Stack.
2. Cuando encuentra una operación asíncrona (como un setTimeout, fetch o Promise), la delega a las Web APIs del navegador.
3. Mientras la operación asíncrona se ejecuta en segundo plano, JavaScript sigue ejecutando otras instrucciones.
4. Cuando la operación finaliza, la Web API envía el callback a la cola de tareas.
5. El Event Loop revisa constantemente el Call Stack. Cuando este está vacío, mueve las tareas pendientes desde la cola de tareas al Call Stack para ejecutarlas.



Microtasks vs Macrotasks

Hasta ahora sabemos que:

- JS es monohilo
- Tiene Call Stack
- Tiene Task Queue
- El Event Loop mueve tareas cuando la pila está vacía

Pero no todas las tareas asíncronas tienen la misma prioridad.

Macrotasks (Task Queue)	**Microtasks (Microtask Queue)**
• `setTimeout` • `setInterval` • Eventos del DOM • Peticiones HTTP Se ejecutan cuando el Call Stack queda libre.	• `.then()` • `.catch()` • `.finally()` • `await` Las microtasks tienen prioridad sobre las macrotasks.

Microtasks vs Macrotasks

```
console.log("Inicio");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() =>
console.log("Promesa"));
console.log("Fin");
```

```
Inicio
Fin
Promesa
Timeout
```

Las Promesas se ejecutan antes que los setTimeout aunque el timeout sea 0.

¿Cómo funciona el Scroll Infinito con Asincronía?

El scroll infinito es una técnica en la que, a medida que el usuario se desplaza hacia abajo en una página web, se cargan nuevos datos automáticamente sin necesidad de recargar la página.

Para lograrlo, se utiliza asincronía porque:

1. Se ejecuta una solicitud a un servidor (fetch API o AJAX) cuando el usuario llega al final de la página.
2. Mientras se espera la respuesta, el usuario puede seguir interactuando con la página.
3. Una vez que los nuevos datos están listos, se agregan dinámicamente al DOM sin bloquear la ejecución del resto del código.

03

Implementación

Callbacks, Promesas, Fetch, Async/Await

1. Callbacks

Un **callback** es una función que se pasa como argumento a otra función y se ejecuta después de que la función principal termine su trabajo. ➔ *Cuando acabes de hacer lo que estás haciendo haz esto otro.*

Se usaban mucho, pero pueden hacer que el código se vuelva difícil de leer cuando hay muchas llamadas anidadas (callback hell).

```
function obtenerDatos(callback) {
   setTimeout(() => {
       console.log("Datos obtenidos");
       callback();
   }, 2000);
 }
  function mostrarMensaje() {
   console.log("Proceso terminado");
 }
  obtenerDatos(mostrarMensaje);
```

2. Promesas

Las Promesas son la solución a los **Callbacks**.

Como definición, una **Promesa** es la solución moderna que representa un valor que estará disponible ahora, en el futuro o nunca.

Tienen dos estados:

- Resolve (`éxito`) → Se obtiene el resultado correctamente.
- Reject (error) → Hay un error.

Se gestionan con .then(), .catch() y .finally()

```
function obtenerDatos() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            let exito = true; // Cambia a false para simular error
            if (exito) {
                resolve("Datos obtenidos");
            } else {
                reject("Error al obtener datos");
            }
        }, 2000);
    });
}

obtenerDatos()
    .then(mensaje => console.log(mensaje))// Se ejecuta si todo va bien
    .catch(error => console.error(error))// Se ejecuta si hay un error
    .finally(() => console.log("Proceso terminado"));
```

2. Promise.all

Cuando hacemos varias peticiones a la vez hemos de hacer una petición concurrente.:

```
const names = ["pikachu", "charizard", "gengar", "lucario"];

const pokemons = await Promise.all(
 names.map(name => getPokemon(name))
);

console.log(pokemons);
```

Todas las peticiones se lanzan al mismo tiempo.

Esto es clave en backend cuando:

- Consultamos varias tablas
- Llamamos a varios servicios externos
- Construimos respuestas complejas

3. Async/Await

Async/Await es la evolución de las **Promesas**, haciéndolas más fáciles de leer y escribir. Permite escribir código asíncrono con apariencia síncrona, facilitando enormemente la lectura. **Await** pausa la ejecución hasta que la **Promesa** se resuelva.

```
async function obtenerAlumnos() {
    try {
        let response = await fetch("https://itb.cat/alumnos"); // Esperar la respuesta
        let data = await response.json(); // Convertir a JSON, objeto
        console.log("Alumnos obtenidos:", data);
    } catch (error) {
        console.error("Error al obtener datos", error);
    }
}

obtenerAlumnos();
```

04

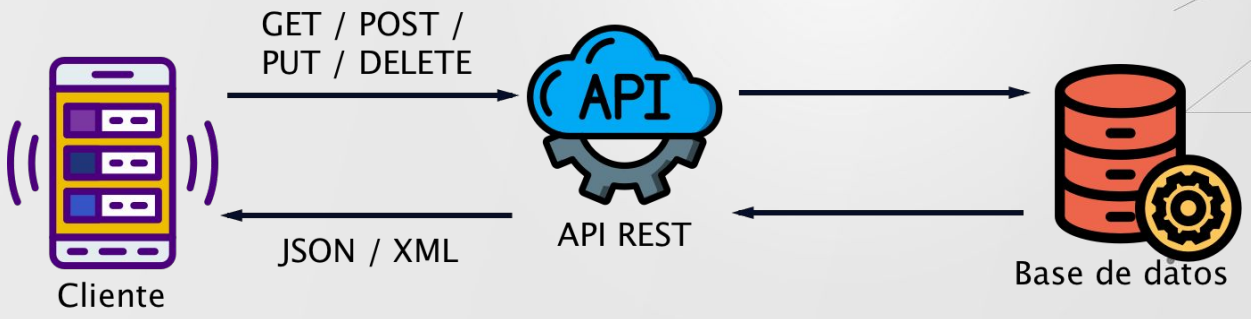
API REST

API REST

API, *Application Programming Interface,* es un conjunto de reglas bien definidas que se utilizan para especificar formalmente la comunicación entre dos componentes de software.

Una **API REST** es una interfaz de comunicación entre sistemas de información que usa el protocolo de transferencia de hipertexto, **HTTP**, para obtener datos o ejecutar operaciones sobre dichos datos en diversos formatos, como pueden ser XML o JSON.

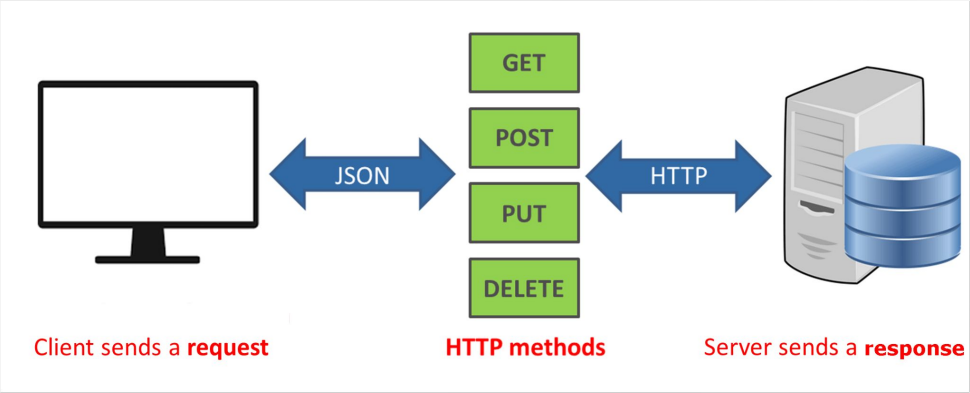
Se basa en el modelo **cliente-servidor** donde el cliente es el que solicita obtener los recursos o realizar alguna operación sobre dichos datos, mientras que el servidor es aquel ente que entrega o procesa dichos datos a solicitud del cliente.

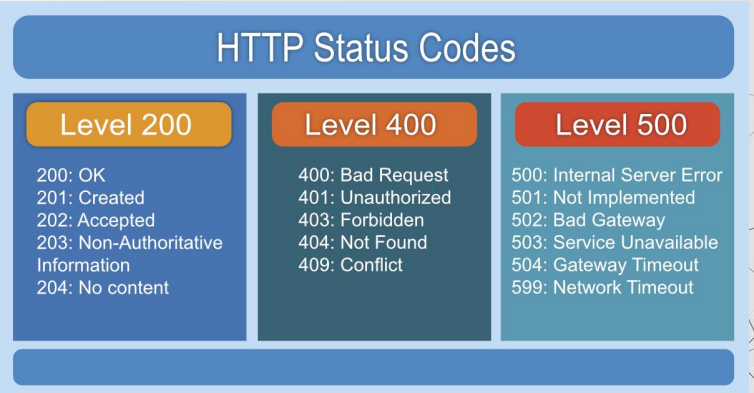


API REST

La **API REST**, interfaz de comunicación que usa el protocolo HTTP para intercambiar datos (normalmente JSON) entre un cliente y un servidor.

- **Métodos**: **GET** (obtener), **POST** (crear), **PUT** (actualizar), **DELETE** (borrar).
- **Status Codes**: 200 (OK), 400 (Errores cliente), 500 (Errores servidor)





05

Fetch

FETCH

Fetch es el nombre de una nueva API para Javascript con la cual podemos realizar peticiones HTTP asíncronas utilizando promesas

A la petición **Fetch** le pasamos la URL como parámetro, a veces necesita más parámetros y esperamos una respuesta, una promesa con el resultado.

```
const promise = fetch("la url");
promise.then(function(response) {
 /* ... */
});
```

```
const options = {
   method: "GET",
   cache: "default"
 };
  fetch("la URL", options)
   .then(response => response.text())
   .then(data => {
     /** Procesar los datos **/
   });
```

FETCH

Se pueden especificar varias opciones según las necesidades de la API y de nuestra acción.

Campo	Descripción
STRING `method`	Método HTTP de la petición. Por defecto, `GET`. Otras opciones: `HEAD`, `POST`, etc...
OBJECT `headers`	Cabeceras HTTP. Por defecto, `{}`.
OBJECT `body`	Cuerpo de la petición HTTP. Puede ser de varios tipos: `String`, `FormData`, `Blob`, etc...
STRING `credentials`	Modo de credenciales. Por defecto, `omit`. Otras opciones: `same-origin` e `include`.

La respuesta

La respuesta es un objeto que tienes varias propiedades dónde nos indica si la respuesta ha ido o no bien.

Propiedad	Descripción
NUMBER .status	Código de error HTTP de la respuesta (100-599).
STRING .statusText	Texto representativo del código de error HTTP anterior.
BOOLEAN .ok	Devuelve true si el código HTTP es 200 (o empieza por 2).
OBJECT .headers	Cabeceras de la respuesta.
STRING .url	URL de la petición HTTP.

La respuesta

En los métodos **GET**, que ahora mismo son los que nos interesan, nuestra información vendrá dentro de un JSON que deberemos procesar para poder mostrar a nuestros usuarios en la página web. Esto a su vez devuelve una nueva promesa

Método	Descripción
STRING `.text()`	Devuelve una promesa con el texto plano de la respuesta.
OBJECT `.json()`	Idem, pero con un objeto `json`. Equivale a usar `JSON.parse()`.
OBJECT `.blob()`	Idem, pero con un objeto `Blob` (binary large object).
OBJECT `.arrayBuffer()`	Idem, pero con un objeto `ArrayBuffer` (buffer binario puro).
OBJECT `.formData()`	Idem, pero con un objeto `FormData` (datos de formulario).
OBJECT `.clone()`	Crea y devuelve un clon de la instancia en cuestión.
OBJECT `Response.error()`	Devuelve un nuevo objeto `Response` con un error de red asociado.
OBJECT `Response.redirect(url, code)`	Redirige a una `url`, opcionalmente con un `code` de error.

```javascript
function conectaAPI(){
  fetch("https://api.chucknorris.io/jokes/random")
      .then(res => {
          if (!res.ok) {
              throw new Error(`Error HTTP ${res.status}`);
          }
          return res.json();
      })
      .then(json => {
          muestraChiste(json.value);
      })
      .catch(error => {
          console.error("Error:", error);
      });
}

//-----------------------------------------------------------------
//Versión moderna con async/await
async function conectaAPI() {
   try {
       const res = await fetch("https://api.chucknorris.io/jokes/random");
       if (!res.ok) {
           throw new Error(`Error HTTP ${res.status}`);
       }
       const json = await res.json();
       muestraChiste(json.value);
   } catch (error) {
       console.error("Error:", error);
   }
}
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
content="width=device-width,
initial-scale=1.0">
   <title>API REST</title>
   <script src="ApiRest.js" defer></script>
</head>
<body onload="conectaAPI()">
   <div id="chistes"></div>
   <button id="masChistes">Frase de
Chuck</button>
</body>
</html>
```

Ejercicio1: PokeAPI

Crear una pequeña aplicación que:

1. Se conecte a la api y muestre el listado de los pokemons
2. Cargue 4 Pokémon

   Muestre:

   - Nombre
   - Imagen
   - Tipos
   - ID
   - Controle errores

3. Permita buscar un Pokémon por nombre